

Cẩm nang đọc source và lần theo pipeline VideoLevel

Từ điểm vào demo, CLI và API đến các hàm native C++

Tài liệu kỹ thuật tiếng Việt

05/10/2026

Cập nhật 05/10/2026. Tài liệu được viết lại theo cây mã nguồn hiện tại. Repository chỉ có **một lỗi media** là thư viện C++ native/src/cavlc_stream.cpp. Kênh dùng **giao thức v3** (04/10/2026): khung `[0x03][độ dài 2 byte][payload]` **không có MAC**, hai khóa con HKDF (lịch và làm trắng), một giao thức duy nhất theo đoạn IDR. Proof Groth16 dùng mạch mới `circuits/camera_video.circom`: *một camera trong sổ đăng ký (gốc Merkle Poseidon) xác nhận đúng video này và message này*; bên kiểm chứng chỉ cần root, verification key và khóa giấu tin, không cần secret camera. Endpoint `/api/v1/jobs/verify` là nơi duy nhất xác thực payload (trích mù → hash video → Groth16 bắt buộc). Nguồn đối chiếu: `doc/he_thong_hoat_dong.md`, `README.md`, `benchmark/NEW_BENCHMARKS.md` và chính mã nguồn.

Mục lục

1	Mục đích và cách dùng tài liệu	3
1.1	Ba lớp kết quả cần phân biệt	3
1.2	Hai khóa tách biệt	3
2	Sơ đồ hoạt động tổng thể	3
2.1	Ba điểm vào và nơi chúng gặp nhau	3
2.2	Luồng gọi theo thứ tự trong demo	4
3	Cách đọc cấu trúc repository	5
4	Điểm vào demo: demo/terminal_demo.py	7
4.1	Session và các helper dùng chung	7
4.2	Các module demo và hàm chính	7
5	Pha prover: từ message đến payload mang proof	8
5.1	Mệnh đề mà proof chứng minh	8
5.2	Sổ đăng ký camera	9
5.3	Hash video và binding	9
5.4	Hàm theo thứ tự gọi ở phía gửi	10
5.5	Ví dụ byte-for-byte với demo	11
5.6	Mã hóa/nén proof	11
5.7	Xác minh Groth16 và setup	11

6 Đường native: đọc H.264 và liệt kê ứng viên dấu CAVLC	11
6.1 Tại sao lật đúng một bit dấu giữ parser đồng bộ	11
6.2 Annex-B, EBSP và RBSP	11
6.3 Các kiểu dữ liệu C++ giữ kết quả parser	12
6.4 Call chain C++ từ Annex-B đến ứng viên	12
6.5 Ví dụ một ứng viên	13
7 Kênh giấu tin v3: khóa con, lịch, khung và làm trắng	14
7.1 Khóa con HKDF	14
7.2 Lịch có khóa	14
7.3 Khung kênh và làm trắng	14
7.4 Giao thức theo đoạn IDR	15
7.5 Thông báo lỗi native	16
8 CLI native và công cụ soi	16
8.1 zkstego_blind_bits	16
8.2 zkstego_inspect và lịch tham chiếu Python	18
9 Verifier: từ video stego đến quyết định	18
9.1 Trích mù native	18
9.2 Unpack, hash video và verify Groth16	19
9.3 Verifier phía API	19
10 API: hàm nào nhận request, hàm nào chạy job	19
10.1 Lỗi job chung src/api/app.py	19
10.2 Ứng dụng native src/api/native_handlers.py	21
11 Các module hỗ trợ ngoài đường embed/extract	22
11.1 Bộ lập lịch realtime	22
11.2 Chứng chỉ khóa và manifest	22
11.3 Giao diện trust thử nghiệm	22
12 Kiểm thử và tái lập	22
12.1 Số liệu hiện tại để đối chiếu	23
13 Demo đã chạy: artifact và cách suy luận	24
14 Các giới hạn cần nhớ khi đọc code	24
15 Lộ trình đọc source cho người mới tiếp nhận	25
16 Tra cứu nhanh theo câu hỏi debug	25
17 Kết luận vận hành	26

1 Mục đích và cách dùng tài liệu

Tài liệu này là bản đồ đọc code của repository VideoLevel. Mục tiêu là giúp người đọc lần theo một dữ liệu cụ thể qua từng lớp: điểm vào (demo, CLI, API), bộ sinh số đăng ký camera, hash video, proof Groth16, CLI native, thư viện C++ phân tích CAVLC, lịch nhưng có khóa, khung kênh, rồi ngược lại ở phía bên nhận. Mỗi phần nêu file, hàm, đầu vào, đầu ra, tác dụng lên pipeline và bước kế tiếp dùng kết quả đó. Mọi tên file và tên hàm trong tài liệu đã được đối chiếu với mã nguồn ngày 05/10/2026. Đây là hướng dẫn source, không phải đặc tả cho mọi profile H.264.

Điểm quan trọng đầu tiên: chỉ có **một** đường xử lý video. Demo, HTTP job và WebSocket đều gọi cùng tiến trình `zkstego_blind_bits`, và tiến trình này gọi cùng các hàm trong `native/src/cavlc_stream.cpp`; kể cả hash ràng buộc video (lệnh `video-digest`) cũng do native tính. Python chỉ điều phối: dựng số đăng ký, tính binding, sinh/verify proof, đóng gói payload, quản lý job/stream và giải thích trong demo. Python có bản tham chiếu byte-for-byte của lịch và khung (`src/native_blind_contract.py`) và của hash video (`src/video_binding.py`) để kiểm thử chéo, nhưng các bản này không nhúng hay trích video.

1.1 Ba lớp kết quả cần phân biệt

- **Thành công cú pháp video:** FFmpeg đọc và giải mã stream mà không báo lỗi nghiêm ngặt. Điều này chưa đo PSNR, SSIM hay chất lượng cảm nhận.
- **Tìm thấy khung kênh:** extractor native đọc bit dấu theo lịch có khóa, bỏ làm trắng, và byte version (0x03) cùng trường độ dài hợp lệ. Điều này cần đúng khóa giấu tin 32 byte và đúng `max_bits_per_IDR`. Khung v3 không có MAC, nên payload lúc này chỉ là *ứng viên* ("`verified`": false).
- **Thành công mệnh đề ZK:** payload unpack đúng, hash video tính lại từ chính file nhận được, và `snarkjs groth16 verify` trả mã thoát 0 và in OK! trên public signals [`root`, `bindingHi`, `bindingLo`]. Kết quả này chứng minh một camera trong số đăng ký đã xác nhận đúng video và message; nó không chứng minh camera thật sự quay cảnh đó (ví dụ quay lại màn hình).

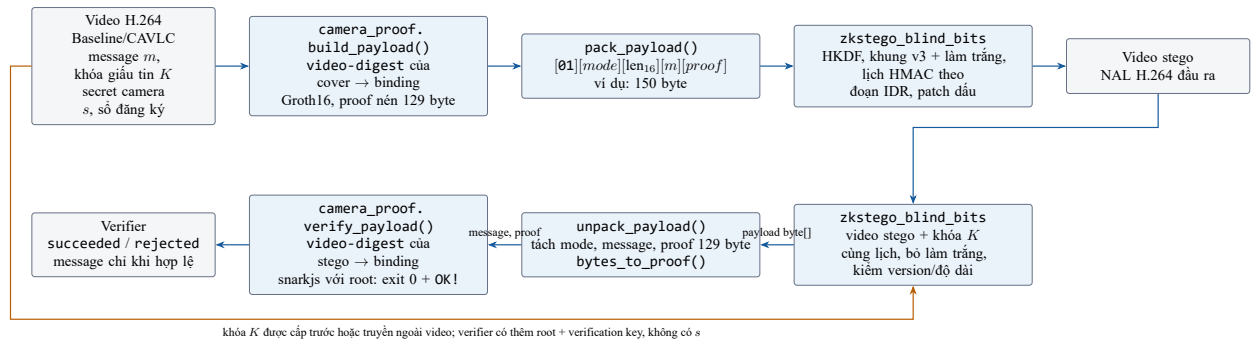
1.2 Hai khóa tách biệt

Khóa	Ai giữ	Dùng để
Khóa giấu tin K (32 byte)	Người gửi và bên kiểm chứng	Chọn vị trí nhúng, làm trắng bit, xác định các bit carrier trong hash video.
Secret camera s (phần tử trường BN254)	Chỉ camera	Tạo proof; số đăng ký chỉ lưu $pk = \text{Poseidon}(s)$.

Verifier vẫn cần K vì K sinh lại các khóa con để tái tạo lịch và bỏ làm trắng, và vì hash video phụ thuộc vị trí carrier do K quyết định. Nếu verifier đã được cấp khóa an toàn từ trước thì kênh media chỉ cần truyền video; nếu chưa, cần một kênh cấp khóa riêng. Verifier **không** cần secret camera và không biết camera nào trong số đăng ký đã tạo proof; nó chỉ tin gốc Merkle (root) và verification key. Endpoint `/api/v1/jobs/verify` thực hiện trích mù → unpack → hash video của file tải lên → Groth16 verify với root, và chỉ trả message khi proof hợp lệ.

2 Sơ đồ hoạt động tổng thể

2.1 Ba điểm vào và nơi chúng gặp nhau



Hình 1: Luồng proof-bearing. Khóa giấu tin là input ngoài video ở cả pha nhúng lẫn trích mù; secret camera chỉ ở phía camera.

Điểm vào	Hàm Python/C++ đầu tiên	Gặp lỗi native ở đâu
Demo	demo/terminal_demo.py:: main() → walkthrough()	demo/native_io.py:: native_embed() / native_extract() chạy zkstego_blind_bits; trace_video() / inspect_segments() chạy zkstego_inspect.
CLI	native/tools/blind_bits. cpp::wmain()/main() → run()	Gọi trực tiếp embed_cavlc_stream_file(), extract_cavlc_stream_ file(), cavlc_video_binding_ digest() hoặc các hàm live trong cùng file.
Proof camera	src/camera_proof.py:: build_payload() / verify_payload()	video_binding.native_video_ digest() chạy zkstego_blind_ bits video-digest (khóa qua stdin).
API HTTP	src/api/app.py:: start_embed()/start_ extract()/start_verify()	Handler do src/api/native_handlers. py::create_native_app() inject; invoke() chạy CLI với khóa qua stdin; job verify gọi thêm camera_proof.check_proof().
API WebSocket	native_handlers.py:: native_stream()	Mở tiến trình con embed-live-auth-stdin hoặc extract-live-auth-stdin.
Soi/kiểm toán	native/tools/inspect.cpp:: run()	Chỉ đọc: decode_baseline_i_ idr_slices(), analyze_cavlc_ stream_file().

2.2 Luồng gọi theo thứ tự trong demo

walkthrough() là kịch bản chính; mỗi session.stage() in một bước trên terminal. Bảng dưới liệt kê đúng thứ tự gọi trong mã.

#	Hàm được gọi	Input chính	Output chuyển tiếp
1	setup() (chỉ khi --setup)	Toolchain CMake, npm, Circom, PTAU Hermez	Binary native, artifact mạch, khóa Groth16 <i>demo</i> qua <code>src.zk_setup</code>
2	preflight()	PATH, circuits/build	Tools, MeasuredBridge; dừng nếu thiếu artifact
3	video_prepare() (choose_input())	File .y4m/video, --frames, --gop, --qp	<code>source.h264</code> Baseline/CAVLC, <code>encode.stderr</code>
4	choose_message(); secrets.token_bytes(32)	Message UTF-8	message bytes; khóa giấu tin ngẫu nhiên 32 byte
5	trace_video() + show_video_trace()	<code>source.h264</code>	<code>trace_before.json</code> (NAL, slice, ứng viên)
6	deep_trace.nal_anatomy(), library_header_trace()	Trace + FFmpeg <code>trace_headers</code>	Đối chiếu header (bỏ khi --brief)
7	choose_cap()	source, trace, độ dài message	<code>max_bits</code> , JSON <code>segments-1</code>
8	make_camera_proof()	source, message, khóa, <code>max_bits</code>	Sổ đăng ký 4 camera, digest cover, binding, proof 129 byte, DemoCamera, <code>camera_registry.json</code>
9	pack_and_select()	message, proof, khóa, đoạn	payload, EmbedPlan, <code>schedule.json</code>
10	native_embed()	cover, payload, khóa, <code>max_bits</code>	<code>stego.h264</code>
11	trace_video() + audit_embedding()	<code>stego.h264</code>	<code>trace_after.json</code> , ứng viên mẫu
12	visual_quality()	source, stego	PSNR Y/U/V, bản đồ ΔY
13	deep_trace.block_anatomy(), embedded_bit_location()	Ứng viên mẫu, ảnh chưa deblock	Giải phẫu một khối, vị trí byte trong file
14	verify_message()	<code>stego.h264</code> , khóa, root	digest stego = digest cover; <code>verify_payload()</code> hợp lệ, <code>video_bound</code>
15	negative_cases.run_all()	stego, khóa sai, proof sửa, video cắt	Các ca từ chối đúng lý do
16	service_steps.run_all()	cùng input + <code>camera_registry.json</code>	Job HTTP và WebSocket (bỏ khi --no-service)
17	Session.finish()	Trạng thái	<code>transcript.txt</code> , <code>report.json</code>

3 Cách đọc cấu trúc repository

Đường dẫn	Vai trò và ảnh hưởng đến pipeline
native/include/zkstego/cavlc_stream.hpp	Kiểu dữ liệu và interface công khai: SPS/PPS/slice, macroblock, ứng viên, HKDF, lịch, khung, stream encoder/decoder, reader Annex-B tăng dần.
native/src/cavlc_stream.cpp	Lỗi media duy nhất: parser Annex-B/SPS/PPS/IDR/CAVLC, ứng viên đầu, khóa con HKDF, lịch HMAC, khung v3, làm trắng, patch RBSP, chế độ file và chế độ stream, hash ràng buộc video.
native/tools/blind_bits.cpp	CLI zkstego_blind_bits: đọc khóa/payload qua stdin, dispatch 6 lệnh, ghi H.264, payload hex hoặc dòng ZKSTEGO_VIDEO_DIGEST.
native/tools/inspect.cpp	Công cụ soi duy nhất zkstego_inspect: JSON (mặc định), --summary, --macroblock, --segments.
native/tools/cli_io.hpp	I/O dùng chung cho hai tool: đổi số UTF-8 trên Windows, đọc file có giới hạn, ghi file mới không ghi đè.
native/tests/cavlc_stream_tests.cpp	CTest native (zkstego_cavlc_stream_tests).
src/zk_proof.py	Định dạng payload (pack_payload(), unpack_payload()), nén proof 129 byte, CameraProofBridge gọi Node/snarkjs để prove/verify.
src/camera_proof.py	Phía gửi build_payload() và phía nhận verify_payload(), check_proof(), parse_payload(), Verdict.
src/video_binding.py	Bản tham chiếu hash video (video_digest()), native_video_digest() gọi CLI, binding_digest(), binding_public_inputs().
src/camera_registry.py, src/poseidon.py	Sổ đăng ký camera: cây Merkle Poseidon độ sâu 16 (CameraRegistry); Poseidon Python với hằng số circomlib (src/poseidon_constants.json).
circuits/camera_video.circom	Mạch CameraVideo(16) + MerkleInclusion: 8 737 constraint, 3 public input [root, bindingHi, bindingLo].
src/zk_setup.py	Setup Groth16: Phase 1 Hermez Powers of Tau 2 ¹⁶ , Phase 2 nhiều lượt đóng góp + beacon + snarkjs zkey verify.
src/native_blind_contract.py	Bản tham chiếu Python của HKDF, lịch, khung, làm trắng và segment_schedule(); dùng cho test chéo và phần giải thích của demo.
src/api/app.py	Lỗi job chung: settings, xác thực trước body, giới hạn thử sai, hàng đợi, lưu trữ job, endpoint.
src/api/native_handlers.py	Ứng dụng native: handler embed/extract/verify gọi CLI, verify Groth16 bắt buộc, WebSocket /api/v1/stream.
src/realtime_cavlc.py	Bộ lập lịch realtime mô phỏng theo đoạn IDR (hàng đợi có giới hạn, epoch, p95).
src/h264_tables.py	Chỉ chứa bảng VLC CAVLC chuẩn cho phần giải thích của demo; không có parser.
src/key_policy.py, src/manifest.py	Chúng chỉ khóa Ed25519 có hạn và manifest có chữ ký. Không nằm trên đường embed/extract native.
src/trust/	Giao diện trust thử nghiệm: provenance, fingerprint, attestation, watermark receipt, c2pa_bridge.py.

Đường dẫn	Vai trò và ảnh hưởng đến pipeline
demo/	Demo duy nhất <code>terminal_demo.py</code> và các module <code>deep_trace.py</code> , <code>h264_explain.py</code> , <code>segment_plan.py</code> , <code>negative_cases.py</code> , <code>service_steps.py</code> , <code>native_io.py</code> .
src/runtest/	<code>run_all.py</code> (12 phase), <code>prepare_fixtures.py</code> , các file <code>test_*.py</code> .
benchmark/	Benchmark media/crypto/ZKP, bộ ghi benchmark camera và báo cáo.

4 ĐIỂM VÀO demo: demo/terminal_demo.py

4.1 Session và các helper dùng chung

Mọi bước đều đi qua đối tượng `Session` trong `terminal_demo.py`. Thư mục chạy là `demo/runs/terminal_<YYYYmmdd_HHMMSS>_<6 hex>/`.

Phương thức	Input → output	Vai trò
<code>Session.say()</code>	text, private → terminal + transcript	Giá trị <code>private=True</code> (secret, khóa con) chỉ in ra terminal; transcript ghi dòng che.
<code>Session.stage()</code>	tiêu đề → bước mới	Dừng chờ Enter nếu không có <code>--auto</code> .
<code>Session.run()</code>	label, argv, stdin → CompletedProcess	Chạy tiến trình con, lưu <code><label>.stdout/.stderr</code> và thời gian.
<code>Session.check()</code>	tên, bool → [PASS]/[FAIL]	Một [FAIL] dừng demo với mã thoát 1.
<code>Session.save()</code>	tên, giá trị → file JSON	Lưu trace, lịch, proof vào thư mục chạy.
<code>Session.finish()</code>	—	Ghi <code>report.json</code> , đóng transcript.

4.2 Các module demo và hàm chính

Hàm	Tác dụng
<i>demo/native_io.py — mọi lời gọi tool native</i>	
<code>native_tool()</code> , <code>find_tools()</code> <code>trace_video()</code>	Tìm <code>zkstego_blind_bits</code> và <code>zkstego_inspect</code> trong <code>native/build/Release</code> , <code>Debug</code> hoặc <code>native/build</code> .
<code>inspect_segments()</code>	<code>zkstego_inspect <video> → JSON schema 1 (NAL, slice IDR, ứng viên toàn file).</code>
<code>native_embed()</code>	<code>embed-stream-auth-stdin</code> ; stdin = khóa hex + payload hex.
<code>native_extract()</code>	<code>extract-stream-auth <video> - 4096 MAX</code> ; stdin = khóa hex.

Hàm	Tác dụng
<i>Ngoài native_io.py: hash video và verify</i>	
src.video_binding. native_video_digest()	video-digest <video> FRAME_BITS MAX; stdin = khóa hex; demo gọi cho cover (trước khi prove) và cho stego (khi verify).
src.camera_proof.verify_payload()	Groth16 qua CameraProofBridge.verify(); chỉ hợp lệ khi snarkjs exit 0 và có OK!; lỗi hạ tầng ném RuntimeError, không bị tính là “từ chối hợp lệ”.
<i>demo/segment_plan.py — dung lượng và lịch theo đoạn</i>	
resolve_max_bits()	Mặc định 64 bit/IDR; nếu không truyền cò và clip quá ngắn thì nâng lên giá trị nhỏ nhất đủ dung lượng.
segment_capacity(), show_allocation()	Tính $\sum \min(\text{max_bits}, \text{ứng viên})$; in đoạn nào mang khoảng bit nào của khung.
file_candidate()	Đổi định danh ứng viên trong đoạn sang chỉ số NAL trong file.
<i>demo/deep_trace.py và demo/h264_explain.py — giải thích sâu</i>	
ffmpeg_pipeline(), raw_input_view(), encode_loss()	Đọc log verbose của Ffmpeg/libx264, hiển thị frame thô, PSNR do nén.
nal_anatomy(), library_header_trace()	Bảng NAL/EPB, đối chiếu header do trace_headers của Ffmpeg giải với parser native.
block_anatomy()	Gọi zkstego_inspect --macroblock, decoder_mb_maps(), rồi các hàm h264_explain (parse_i_macroblock_header(), segment_cavlc_block(), dequantize(), inverse_transform(), predict_intra4x4())... để tái tạo pixel từng mẫu.
embedded_bit_location(), file_offset_of_rbsp_bit()	Từ thứ hạng HMAC → chỉ số bit khung → trường payload → byte RBSP/EBSP → offset trong file.
<i>demo/negative_cases.py và demo/service_steps.py</i>	
negative_cases.run_all()	Đổi message, proof sai, thiếu proof, sai khóa, lật một bit carrier của message (_flip_one_carrier_bit()), cắt NAL cuối (_cut_last_frame()), camera ngoài sổ đăng ký (_unregistered_camera()); lỗi native nhận diện bằng is_frame_not_found(), còn lại kiểm reason của verify_payload().
service_steps.run_all()	Dùng create_native_app() trong TestClient; _http_jobs() (embed/extract/verify) và _websocket_stream().

5 Pha prover: từ message đến payload mang proof

5.1 Mệnh đề mà proof chứng minh

Mạch circuits/camera_video.circom (template CameraVideo(16)) chứng minh: “một camera có khóa trong sổ đăng ký (gốc Merkle công khai) xác nhận đúng binding này”, trong đó binding cam kết hash của video và hash của message.

```

private: secret s; siblings[16]; pathIndices[16]
public : root, bindingHi, bindingLo (3 public inputs)
pk = Poseidon(s) (Poseidon(1))
MerkleInclusion(16): pathIndices[i]*(pathIndices[i]-1) = 0
                    node[i+1] = Poseidon(left[i], right[i])
                    membership.root === root
bindingHiSquare <= bindingHi*bindingHi; bindingLoSquare <= bindingLo*bindingLo

```

Mạch có 8 737 constraint. Hai tín hiệu bình phương chỉ để gắn `bindingHi`, `bindingLo` vào R1CS một cách tường minh (kiểu Semaphore). Proof tạo cho một binding (video + message) hay một root khác đều không verify được. Verifier không cần s và không biết lá nào (camera nào) đã được dùng.

Ghi chú lịch sử. Trước 04/10/2026 repository dùng mạch SHA-256 `payload_verify.circom` (63 321 constraint, 513 public input), trong đó verifier phải biết secret để tính lại commitment và proof không ràng buộc video. Mạch đó đã bị gỡ; proof tạo bằng khóa của mạch cũ không verify được với khóa hiện tại.

5.2 Sổ đăng ký camera

`src/camera_registry.py` dựng cây Merkle nhị phân độ sâu 16 (`TREE_DEPTH`, tối đa 65 536 camera), nút Poseidon(trái, phải), lá trống bằng 0, lá i là $pk_i = \text{Poseidon}(s_i)$. Poseidon cài bằng Python (`src/poseidon.py`, hằng số `circmlib` trong `src/poseidon_constants.json`) và được phase 1 đối chiếu với vector của `circmlibjs` và với witness của chính mạch.

Hàm/lớp	Vai trò
<code>new_camera_secret()</code>	Phần tử trường BN254 ngẫu nhiên khác 0.
<code>camera_public_key(s)</code>	<code>Poseidon([s])</code> , giá trị đăng ký vào cây.
<code>CameraRegistry(public_keys)</code>	Dựng mọi tầng của cây; từ chối khóa trùng, khóa ngoài trường hoặc vượt 2^{16} lá; thuộc tính root.
<code>index_of()</code> , <code>path(index)</code>	Vị trí lá và MerklePath (<code>siblings</code> , <code>path_indices</code>) làm private input.
<code>to_json()/from_json()</code> , <code>save()/load()</code>	Schema <code>zkstego-camera-registry-1</code> ; <code>from_json()</code> kiểm root khớp các khóa công khai.
<code>save_camera_secret()</code> , <code>load_camera_secret()</code>	File secret mới, chỉ chủ sở hữu đọc được, không ghi đè; schema <code>zkstego-camera-secret-1</code> .

5.3 Hash video và binding

```

frame_bits = 8 * (3 + payload_size(len(m))) # camera_proof.
frame_bits_for_message()
carrier = the frame_bits sign positions the keyed schedule (stego key K) will
write
        (per IDR: first min(cap, candidates) schedule positions, until the
        frame ends)
digest = SHA256("zkstego/video-digest/v1" || u32be(frame_bits) || u32be(cap) ||
        for every NAL in file order: u32be(1 + len(rbsp')) || header ||
        rbsp')
rbsp' = NAL RBSP (EPB removed) with ONLY the carrier sign bits cleared to 0
binding = SHA256("zkstego/proof-binding/v1" || mode || digest || SHA256(m))
bindingHi, bindingLo = big-endian halves of binding (two 128-bit field elements)

```

Bản chạy thật là `cavlc_video_binding_digest()` trong `native/src/cavlc_stream.cpp` (lệnh CLI `video-digest`); nó duyệt file bằng `visit_cavlc_stream_file()`, dựng từng đoạn bằng

`prepare_cavlc_stream_segment()` và chọn vị trí bằng `select_with_schedule_key()`, đúng như encoder. Bản tham chiếu Python `video_binding.video_digest()` dùng `segment_schedule()` trên JSON `segments-1`. `native_video_digest()` chạy CLI với khóa qua stdin và đọc dòng `ZKSTEGO_VIDEO_DIGEST <hex> carrier_segments=N nal_units=M`.

Những chỉ đổi đúng các bit carrier (và có thể thêm/bớt byte EPB, không chạm RBSP), nên **digest của cover bằng digest của stego**. Mọi thay đổi khác (video khác, cắt hay thêm frame, sửa một bit ở frame sau, lật một dấu không mang khung, mã hóa lại) làm digest đổi và proof sai. Lật một bit carrier thì digest giữ nguyên nhưng payload trích ra đổi, nên proof cũng bị từ chối. Vị trí carrier phụ thuộc khóa giấu tin, vì vậy lệnh `video-digest` cần khóa (một dòng hex trên stdin).

`binding_digest()` nhận mode: 0 = ràng buộc video (luồng file); 1 = chỉ ràng buộc message, digest bắt buộc là 32 byte 0, dành cho luồng live vì khi những proof thì phần còn lại của video chưa tồn tại. Job verify từ chối mode 1 với lý do `video_binding_required`, trừ khi bật `ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1`.

5.4 Hàm theo thứ tự gọi ở phía gửi

Hàm	Input → output	Tác dụng kế tiếp
<code>frame_bits_for_message(n)</code>	$n \rightarrow 8(3 + 4 + n + 129)$	Biết số bit khung trước khi có proof (proof luôn 129 byte).
<code>native_video_digest(cli, cover, K, frame_bits, cap)</code>	<code>cover, K</code> → digest 32 byte	Đầu vào của binding.
<code>binding_digest(mode, digest, m)</code>	→ binding 32 byte	<code>binding_public_inputs()</code> tách hai nửa 128 bit.
<code>CameraProofBridge.circuit_input()</code>	<code>s</code> , số đăng ký, binding → dict	<code>secret, siblings, pathIndices, root, bindingHi, bindingLo</code> .
<code>_compute_witness(input)</code>	input → file witness tạm	WASM của mạch; <code>input.json</code> chứa <code>secret</code> camera bị xóa trong <code>finally</code> ; thư mục <code>zpk_witness_*</code> còn sót do tiến trình bị giết được <code>_remove_stale_temp_dirs()</code> dọn sau 1 giờ.
<code>_snarkjs_prove(witness)</code>	witness + <code>camera_video.zkey</code> → (proof, public)	Thư mục tạm bị xóa sau khi đọc kết quả.
<code>prove(s, registry, binding)</code>	→ proof dict	Kiểm public signals trả về đúng <code>public_signals(root, binding)</code> .
<code>proof_to_bytes(proof)</code>	A, B, C → 129 byte	Nén proof để nhúng.
<code>pack_payload(mode, m, proofBytes)</code>	→ <code>[01][mode][len_{16BE}][m][proof]</code>	Blob opaque đưa cho CLI native.
<code>build_payload(...)</code>	bridge, số đăng ký, <code>s, m</code> , cover, <code>K</code> → <code>SignedPayload</code>	Gói toàn bộ chuỗi trên; trả payload, mode, digest, binding, proof.

5.5 Ví dụ byte-for-byte với demo

Với `--auto` và không truyền `--message`, `choose_message()` dùng message Xin chào ZK video, UTF-8 dài 17 byte. `pack_payload(MODE_VIDEO, ...)` ghép:

```
01 | 00 | 00 11 | 58 69 6e 20 63 68 61 6f 20 5a 4b 20 76 69 64 65 6f | <129 proof
    bytes>
fmt  mode len=17                      message UTF-8
```

Tổng payload là $4 + 17 + 129 = 150$ byte. Khung native v3 chỉ thêm 3 byte header, nên khung dài 153 byte, tức 1 224 bit phải nhúng. Trong lần chạy E2E ngày 04/10/2026, payload 201 byte (message 68 byte) cho khung 204 byte = 1 632 bit, chiếm 26 IDR ở 64 bit/IDR. Message tối đa là $4096 - 4 - 129 = 3963$ byte vì lệnh stdin nhận payload tối đa 4096 byte.

5.6 Mã hóa/nén proof

`proof_to_bytes()` ghi tọa độ x của A, hai phần tử $\mathbb{F}_p$ của B.x và C.x (mỗi phần 32 byte big-endian), cộng một byte cờ chứa 3 bit chẵn/lẻ của y : $4 \times 32 + 1 = 129$ byte. `bytes_to_proof()` làm chiều ngược: kiểm tọa độ $< p$, giải phương trình đường cong BN254 để dựng lại y , kiểm điểm nằm trên đường cong; dữ liệu hỏng ném `ValueError`. `unpack_payload()` đòi đúng byte format `0x01`, mode 0 hoặc 1 và độ dài chính xác $4 + |m| + 129$; mọi bộ cục khác ném `ValueError`.

5.7 Xác minh Groth16 và setup

`CameraProofBridge.verify(proof, root, binding)` dùng `public_signals() = [root, bindingHi, bindingLo]` rồi `_snarkjs_verify()` ghi proof/public tạm và chạy `snarkjs groth16 verify` với `camera_video_vkey.json`. Kết quả chỉ là True khi **mã thoát 0 và stdout có dòng kết thúc bằng OK!**; exit 0 mà không có OK! bị coi là không hợp lệ.

Setup: `py -3.12 -m src.zk_setup.compile_circuit()` biên dịch mạch; `run_setup()` kiểm Blake2b đã ghim của `powersOfTau28_hez_final_16.ptau` (Phase 1 công khai của Hermez), chạy `groth16 setup`, mỗi lượt `zkey contribute` nhận entropy qua stdin (không qua argv), thêm `zkey beacon` với giá trị ngẫu nhiên sinh cục bộ, đòi `zkey verify in ZKey Ok!`, rồi xuất `camera_video.zkey`, `camera_video_vkey.json` và transcript `circuits/build/camera_video_setup.json`. Mặc định có 2 lượt đóng góp chạy trên một máy, nên đây là nghi thức **demo**; triển khai thật cần các bên độc lập (chỉ cần một bên trung thực là khóa an toàn).

6 Đường native: đọc H.264 và liệt kê ứng viên dấu CAVLC

6.1 Tại sao lật đúng một bit dấu giữ parser đồng bộ

CAVLC residual block mở đầu bằng `coeff_token` mã hóa `TotalCoeff` và `TrailingOnes`. Với mỗi trailing one, stream ghi một `trailing_ones_sign_flag`: bit 0 là +1, bit 1 là -1. Lật bit này chỉ đổi dấu một hệ số; không đổi số hệ số, n_C của các khối sau, bảng VLC được chọn, độ dài mã hay vị trí khối kế tiếp. Vì vậy native patch đúng bit RBSP đó thay vì giải mã frame thành pixel rồi encode lại. Không thể lật tùy tiện một bit trong NAL: nó có thể sửa `coeff_token`, level hoặc header macroblock và làm lệch toàn bộ phần sau.

6.2 Annex-B, Ebsp và Rbsp

`split_annex_b()` quét start code 3/4 byte, cắt elementary stream thành `AnnexBNalUnit` (offset, độ dài start code, header, payload Ebsp). `AnnexBNalUnit::rbsp()` gọi `ebsp_to_rbsp()` để bỏ byte `0x03` đặt sau `00 00`. Parser đọc cú pháp trên RBSP bằng `RbspBitReader` (`read_bits()`, `read_ue()`),

`read_se()`). Khi patch, `patch_annex_b_rbsp_plan()` áp các `FixedLengthBitPatch` theo từng NAL, `rbasp_to_esp()` chèn lại emulation-prevention byte và `assemble_annex_b()` ghép start code + header + payload. Độ dài bit RBSP không đổi, nhưng độ dài byte EBP có thể lệch vài byte.

6.3 Các kiểu dữ liệu C++ giữ kết quả parser

Kiểu trong header	Dữ liệu và consumer tiếp theo
<code>AnnexBNalUnit</code>	Offset, độ dài start code, <code>nal_ref_idc</code> , <code>nal_unit_type</code> , payload EBP; <code>is_idr()</code> , <code>rbasp()</code> .
<code>H264BaselineSps</code>	Profile/level, id, kích thước theo MB, trường frame/POC; <code>validated_picture_macroblock_count()</code> giới hạn kích thước ảnh.
<code>H264BaselinePps</code>	Entropy mode (phải là CAVLC), slice group, QP khởi đầu, cờ deblocking.
<code>H264BaselineIdrSliceHeader</code>	first MB, slice type, PPS id, frame/IDR id, QP delta và <code>data_bit_offset</code> đầu slice data.
<code>CavlcCoeffToken</code>	TotalCoeff, TrailingOnes, <code>sign_bit_offsets</code> , offset level; <code>start_bit_offset</code> và <code>n_c</code> chỉ để chẩn đoán.
<code>CavlcDecodedLumaBlock</code>	Token, levels, tail (total zeros/runs), hệ số tái dựng. Dùng chung cho luma, LumaDC, chroma DC/AC.
<code>H264BaselineIMacroblockHeader</code>	<code>mb_type</code> , CBP, QP delta, 16 mode Intra4x4, offset residual.
<code>CavlcDecodedIMacroblock</code>	Địa chỉ MB, header, <code>luma_dc</code> , 16 khối luma, 2 chroma DC, 8 chroma AC.
<code>CavlcDecodedIdrSlice</code>	NAL index, SPS/PPS/slice header, danh sách MB, offset trailing bits; đầu vào của bộ thu ứng viên.
<code>CavlcSignCandidate</code>	(NAL index, địa chỉ MB, <code>CavlcResidualCategory</code> , chỉ số khối, offset bit RBSP).
<code>CavlcStreamSegment</code>	Byte output, <code>candidate_capacity</code> , <code>bits_embedded</code> , <code>session_complete</code> ; trả về sau mỗi đoạn.
<code>CavlcStreamSegmentCandidates</code>	Với mỗi đoạn IDR: chỉ số NAL trong file, chỉ số NAL trong input phân tích, danh sách ứng viên.

6.4 Call chain C++ từ Annex-B đến ứng viên

Các hàm dưới đây nằm trong `native/src/cavlc_stream.cpp`, interface ở `native/include/zkstego/cavlc_stream.hpp`:

Hàm	Input → output	Ảnh hưởng/consumer
<code>esp_to_rbsp()</code> / <code>rbasp_to_esp()</code>	Byte vector ↔ byte vector	Chuyển giữa byte NAL và bitstream parser/patcher thao tác.
<code>parse_baseline_sps()</code>	SPS RBSP → SPS	Kích thước, cờ cho slice và MB.
<code>parse_baseline_pps()</code>	PPS RBSP → PPS	Kiểm entropy mode và tham số.

Hàm	Input → output	Ảnh hưởng/consumer
<code>parse_baseline_idr_slice_header()</code>	Slice RBSP + SPS/PPS → header	Chỉ ra <code>data_bit_offset</code> .
<code>parse_baseline_i_macroblock_header()</code>	RBSP + offset → header MB	Quyết định cú pháp residual tiếp theo.
<code>parse_cavlc_coeff_token()</code>	RBSP, bit, n_C → token	Chọn bảng VLC theo n_C ; trả các offset bit đầu.
<code>decode_cavlc_non_trailing_levels()</code>	Token + RBSP → levels	Nhảy qua phần level sau bit đầu.
<code>decode_cavlc_residual_tail()</code> (và biến thể luma/chroma DC)	TotalCoeff, offset → total zeros, runs	Đồng bộ vị trí khối kế tiếp.
<code>reconstruct_cavlc_block()</code>	Levels + runs → hệ số	Điền hệ số theo thứ tự quét.
<code>decode_cavlc_luma_block()</code> , <code>decode_cavlc_luma_ac_block()</code> , <code>decode_cavlc_chroma_dc_block()</code> , <code>decode_cavlc_chroma_ac_block()</code>	RBSP + offset + ngữ cảnh → khối	Chuyên biệt theo loại khối.
<code>decode_cavlc_luma_macroblock()</code> / <code>decode_cavlc_luma_ac_macroblock()</code>	CBP, offset, TotalCoeff hàng xóm → 16 khối	Tính n_C theo hàng xóm đã giải mã.
<code>decode_baseline_i_idr_slice()</code>	RBSP + SPS/PPS → một slice	Lập các MB của I-slice.
<code>decode_baseline_i_idr_slices()</code>	Annex-B → các slice IDR	Theo dõi SPS/PPS theo id; dùng cho inspect, đo dung lượng và chuẩn bị đoạn.
<code>collect_cavlc_trailing_one_sign_candidates()</code>	Các slice → ứng viên	Một ứng viên mỗi khối: dấu trailing-one đầu tiên, theo thứ tự LumaDC, Luma4x4, ChromaDC, ChromaAC.
<code>serialize_cavlc_sign_candidate()</code>	Ứng viên → chuỗi ASCII	Định danh để chấm điểm HMAC.
<code>inspect_baseline_idr_headers()</code>	Annex-B → header/khối đầu IDR	Hàm chẩn đoán, không quyết định lịch.

6.5 Ví dụ một ứng viên

Định danh có dạng `nal_index:macroblock_address:category:block_index:rbsp_bit_offset`, ví dụ `24:326:1:6:67801` (category 1 = Luma4x4). Đây là địa chỉ cú pháp, không phải tọa độ pixel. Trong giao thức theo đoạn, `nal_index` tính trong *input phân tích của đoạn* (SPS+PPS+IDR), không phải chỉ số NAL trong file; demo dùng `segment_plan.file_candidate()` để đổi sang chỉ số trong file.

7 Kênh giấu tin v3: khóa con, lịch, khung và làm trắng

7.1 Khóa con HKDF

Khóa giấu tin 32 byte không bao giờ được dùng trực tiếp làm khóa HMAC. Hàm nội bộ `derive_cavlc_subkey()` gọi `hkdf_sha256_extract()` và `hkdf_sha256_expand()` (RFC 5869); hàm công khai `hkdf_sha256()` dùng cho test vector. Từ giao thức v3 chỉ còn **hai** khóa con; nhân HKDF giữ nguyên như v2 nên lịch và dòng làm trắng không đổi:

```
PRK          = HMAC-SHA256(key = "zkstego-cavlc-v2-salt", msg = secret)
schedule_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/schedule", 32)
whitening_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/whitening", 32)
```

Lộ một khóa con không làm lộ secret hay khóa con còn lại. Encoder/decoder xóa secret của caller ngay sau khi dẫn xuất và chỉ giữ khóa con. HMAC ở đây chỉ đóng vai hàm giả ngẫu nhiên có khóa (chọn vị trí, sinh keystream), không còn dùng để xác thực.

7.2 Lịch có khóa

`select_with_schedule_key()` (nội bộ) kiểm từng ứng viên bằng `validate_blind_schedule_candidate()`, từ chối định danh hoặc đích patch (`nal:bit`) trùng lặp, tính

$$\text{score} = \text{HMAC}(\text{schedule_key}, \text{định danh}),$$

rồi `std::partial_sort` theo (score, định danh) để lấy N ứng viên đầu. Nếu N lớn hơn số ứng viên, hàm ném `blind schedule capacity is insufficient`. Các hàm công khai `score_keyed_cavlc_sign_candidate()` và `select_keyed_cavlc_sign_candidates()` nhận secret gốc, tự dẫn xuất `schedule_key` rồi gọi cùng logic. Cũng hàm này chọn các bit carrier mà `cavlc_video_binding_digest()` xóa về 0 khi tính hash video.

7.3 Khung kênh và làm trắng

```
frame          = [0x03][payload_len BE16][payload]                (no MAC, 3-byte
  header)
keystream      = HMAC(whitening_key, uint64_be(0)) || HMAC(whitening_key, uint64_be(1))
  || ...
embedded[i]    = frame_bit[i] XOR keystream_bit[i]                (i global over all segments,
  MSB-first)
```

Hàm (cavlc_stream.cpp)	Vai trò
<code>pack_frame()</code> (nội bộ)	Ghép version 0x03 (kCavlcFrameVersion), độ dài BE16 và payload. Payload tối đa 65 535 byte theo định dạng.
<code>unpack_frame()</code> (nội bộ)	Kiểm version và độ dài $\leq$ giới hạn, đúng kích thước; ném <code>CAVLC frame version is invalid</code> hoặc <code>CAVLC frame length is invalid</code> .
<code>bytes_to_msb_bits()</code> , <code>msb_bits_to_bytes()</code>	Quy ước bit MSB trước cho cả hai phía.
<code>whitening_block()</code>	Khởi keystream thứ $j = \text{HMAC}(\text{whitening_key}, \text{uint64_be}(j))$.
<code>whiten_cavlc_frame_bits()</code>	XOR bit với keystream bắt đầu từ chỉ số bit toàn cục; gọi lại lần nữa để bỏ làm trắng.

Hàm	Vai trò
pack_cavlc_frame(), unpack_cavlc_frame(), cavlc_whitening_ keystream()	Bản công khai; dùng trong CTest và để so với bản Python.

Làm trắng cần thiết vì byte version cố định, trường độ dài nhỏ và cấu trúc proof làm thống kê dấu tại các vị trí được chọn bị lệch. Khung v3 **không có MAC**: khi Groth16 verify đã là bắt buộc, một thẻ xác thực khung chỉ trùng việc (sửa message hay proof thì proof sai), và bỏ nó tiết kiệm 128 bit. Khóa sai vẫn bị loại rê: byte version sau khi bỏ làm trắng gần như ngẫu nhiên, lọt qua cả kiểm version lẫn độ dài với xác suất cỡ $\frac{1}{256} \cdot \frac{\text{giới hạn}}{65536}$; phần còn sót hổng ở bước giải nén proof hoặc bị Groth16 từ chối. Hệ quả: kênh không tự phát hiện bit payload bị sửa, nên payload trích ra chỉ là ứng viên cho tới khi proof verify. Giao thức v3 không đọc được stego v2/v1 (byte version khác).

7.4 Giao thức theo đoạn IDR

Chỉ có một giao thức, dùng chung cho file, job HTTP và luồng live: mỗi đoạn là ngữ cảnh SPS/PPS đã thấy + đúng một IDR; mỗi đoạn mang tối đa `max_bits_per_IDR` bit của khung; chỉ số bit khung chạy liên tục qua các đoạn. Bên nhận phải dùng cùng `max_bits_per_IDR`.

Hàm	Vai trò
update_parameter_set_context()	Giữ đúng một SPS và một PPS; cùng id thì thay, khác id cùng loại thì từ chối.
assemble_cavlc_stream_segment()	Ghép ngữ cảnh SPS/PPS + một IDR thành một đoạn Annex-B.
prepare_cavlc_stream_segment() (nội bộ)	Kiểm đoạn có đúng một IDR và không có P-slice; dựng input phân tích; giải mã slice; thu ứng viên.
CavlcStreamEncoder (constructor)	Dẫn xuất hai khóa con, đóng khung, đổi sang bit, làm trắng toàn bộ một lần; chỉ giữ <code>schedule_key</code> và bit khung đã làm trắng.
CavlcStreamEncoder::process_segment()	Chọn min(<code>max_bits</code> , ứng viên) vị trí theo lịch; chỉ patch phần bit khung còn lại; vị trí thừa ở đoạn cuối giữ dấu gốc; gọi <code>patch_annex_b_rbsp_plan()</code> rồi <code>remove_stream_context_prefix()</code> .
complete(), remaining_bits() CavlcStreamDecoder::consume_segment()	Caller biết khi nào khung đã nhúng hết. extract_keyed_bits_from_candidates() đọc bit dấu theo lịch; bỏ làm trắng với chỉ số toàn cục; sau 24 bit kiểm version và độ dài; đủ bit thì <code>unpack_frame()</code> . Mọi lỗi chuyển decoder sang <code>failed()</code> (fail-closed).
CavlcStreamDecoder::payload()	Trả payload khi khung đã đủ bit và đúng định dạng; nếu chưa: CAVLC stream is incomplete. Payload này chưa được xác thực.
embed_cavlc_stream_file()	Chế độ file: <code>visit_cavlc_stream_file()</code> duyệt NAL; IDR còn cần bit thì qua encoder và <code>single_idr_payload()</code> ; NAL khác và IDR sau khi xong được sao chép nguyên.
extract_cavlc_stream_file()	Chế độ file phía nhận: đưa từng đoạn vào decoder, dùng khi xong.

Hàm	Vai trò
<code>analyze_cavlc_stream_file()</code>	Góc nhìn kiểm toán: ứng viên của từng đoạn đúng như codec thu được; nguồn của <code>zkstego_inspect --segments</code> .
<code>cavlc_video_binding_digest()</code>	Hash ràng buộc video: cùng cách duyệt và cùng lịch như encoder để xác định đúng <code>frame_bit_count</code> bit carrier, xóa chúng về 0 trong RBSP rồi bấm mọi NAL; trả <code>CavlcVideoBindingDigest</code> (digest, carrier_segments, nal_units).

7.5 Thông báo lỗi native

Mọi lỗi của CLI in error: ... ra stderr và thoát mã 2.

Thông báo	Khi nào / hàm phát sinh
CAVLC stream version is invalid	Decoder: byte đầu sau khi bỏ làm trắng khác <code>0x03</code> , thường do sai khóa hoặc stego v2.
CAVLC stream length exceeds configured maximum	Decoder: trường độ dài vượt giới hạn payload.
CAVLC frame version is invalid, CAVLC frame length is invalid	<code>unpack_frame()</code> khi khung sai định dạng.
blind schedule capacity is insufficient	Lịch cần nhiều bit hơn số ứng viên.
IDR stream capacity is insufficient for payload	Embed chế độ file, hoặc video-digest khi video không đủ chỗ cho khung.
IDR stream ended before payload capacity was met	Embed live (embed-live-auth-stdin).
IDR stream ended before payload was complete	Extract chế độ file.
input stream ended before payload was complete	Extract live (extract-live-auth-stdin).

Tìm thấy khung chỉ có nghĩa là có dữ liệu ở đúng vị trí theo khóa; nó **không** chứng minh payload nguyên vẹn. Việc đó thuộc về proof Groth16 (job verify).

8 CLI native và công cụ soi

8.1 zkstego_blind_bits

`native/tools/blind_bits.cpp`: `wmain()` (Windows, đối số wide) hoặc `main()` gọi `zkstego::cli::utf8_arguments()` rồi `run()`. Hàm `run()` dispatch đúng sáu lệnh. Hậu tố `-auth` được giữ cho tương thích và nay chỉ có nghĩa “có khóa”; khung không có MAC.

Lệnh và đối số	stdin / stdout	Hàm lỗi được gọi
embed-stream-auth-stdin in out max_bits	stdin: dòng khóa hex, dòng payload hex; ghi file mới	embed_cavlc_stream_ file()
extract-stream-auth in key - max_payload max_bits	khóa từ file quyền hạn chế hoặc stdin (-); stdout: payload hex (chưa xác thực)	extract_cavlc_stream_ file()
video-digest in frame_bits max_bits	stdin: dòng khóa hex (cần để định vị bit carrier); stdout: ZKSTEGO_VIDEO_DIGEST <hex> carrier_segments=N nal_units=M	cavlc_video_binding_ digest()
measure-live-capacity- stdin max_bits	stdin: Annex-B; stdout: ZKSTEG_CAPACITY_METRICS JSON	run_live_capacity_ measure()
embed-live-auth-stdin max_bits	stdin: khóa, payload, rồi Annex-B; stdout: H.264; stderr: ZKSTEG_STREAM_METRICS	run_live_embed()
extract-live-auth-stdin max_payload max_bits	stdin: khóa, rồi Annex-B; stdout: payload hex	run_live_extract()

Helper trong blind_bits.cpp	Vai trò
read_key_stdin_line(), read_key_file()	Đúng một dòng 64 ký tự hex; bộ đệm SensitiveKeyText bị xóa khi hủy. Khóa không bao giờ đi qua argv.
read_hex_payload_stdin_ line(), decode_hex() parse_bit_count()	Payload hex không rỗng, tối đa 4096 byte; giữ trong SensitiveBytes. Từ chối số âm, tràn, ký tự thừa; các lệnh tự kiểm max_bits > 0.
set_standard_streams_to_ binary()	Bắt buộc trên Windows để stdin/stdout không đổi byte H.264.
is_bare_start_code(), parse_single_stream_nal() run_live_embed()	Bỏ NAL rỗng ở chế độ live; tách đúng một NAL từ reader. Mỗi IDR: assemble_cavlc_stream_segment() → encoder.process_segment() → ghi lại đúng IDR đã patch; SPS/PPS/NAL khác chuyển nguyên; đo p50/p95 thời gian mỗi đoạn.
run_live_extract()	Mỗi IDR: decoder.consume_segment(); trả ngay khi complete().

native/tools/cli_io.hpp cung cấp utf8_arguments(), os_path(), read_binary_file() (giới hạn kMaxAnnexBFileBytes = 1 GiB) và write_new_binary_file() (không ghi đè file đã có). AnnexBNalStreamReader::read_next() chỉ giữ một NAL cộng một chunk input, nhận start code nằm vắt qua ranh giới chunk, mặc định từ chối NAL lớn hơn 16 MiB.

8.2 zkstego_inspect và lịch tham chiếu Python

`native/tools/inspect.cpp::run()` chọn một trong bốn chế độ: `trace()` (mặc định, JSON schema 1: NAL, slice IDR, ứng viên toàn file), `summary()` (`--summary`, văn bản), `trace_macroblock()` (`--macroblock` NAL MB, schema macroblock-detail-1: header mọi MB của slice và từng phần tử CAVLC của một MB) và `segments()` (`--segments` MAX_BITS, schema segments-1, dựa trên `analyze_cavlc_stream_file()`).

Python `src.native_blind_contract.segment_schedule()` nhận JSON segments-1, khóa, số bit khung và `max_bits_per_IDR`, trả danh sách `SchedulePlacement` (đoạn, ứng viên, chỉ số bit khung) đúng như encoder native. Demo dùng hàm này hai lần: trước khi nhúng (`pack_and_select()`) và sau khi verify (`verify_message()`) để chứng tỏ bên nhận tự dựng lại cùng lịch từ video stego.

Hàm trong <code>src/native_blind_contract.py</code>	Tương ứng native
<code>derive_channel_keys(),</code> <code>hkdf_sha256()</code>	<code>derive_cavlc_subkey(),</code> <code>hkdf_sha256()</code>
<code>NativeCavlcCandidate.</code> <code>serialize(),</code> <code>score_candidate(),</code> <code>select_candidates()</code>	<code>serialize_cavlc_sign_candidate(),</code> <code>score_keyed_cavlc_sign_candidate(),</code> <code>select_keyed_cavlc_sign_candidates()</code>
<code>pack_frame(),</code> <code>unpack_frame()</code> (<code>FRAME_VERSION = 3,</code> <code>FRAME_HEADER_BYTES = 3</code>)	<code>pack_cavlc_frame(),</code> <code>unpack_cavlc_frame()</code>
<code>whitening_keystream_bits(),</code> <code>whiten_bits()</code> (= <code>unwhiten_bits</code>)	<code>cavlc_whitening_keystream(),</code> <code>whiten_cavlc_frame_bits()</code>
<code>embedded_frame_bits(),</code> <code>recover_payload()</code>	Bit khung đã làm trắng / chiều ngược
<code>segment_schedule()</code>	Lịch của <code>process_segment()</code> qua mọi đoạn
<code>src/video_binding.py::</code> <code>video_digest()</code>	<code>cavlc_video_binding_digest()</code> (lệnh <code>video-digest</code>)

9 Verifier: từ video stego đến quyết định

Bên kiểm chứng chỉ cần: video stego, khóa giấu tin K , gốc sổ đăng ký (root) mà nó tin, và `camera_video_vkey.json`. Không cần video gốc, file phụ hay secret camera.

9.1 Trích mù native

“Mù” nghĩa là bên nhận không cần video gốc hay file phụ. Với `extract-stream-auth`, native:

1. duyệt NAL, cập nhật ngữ cảnh SPS/PPS, ghép mỗi IDR thành một đoạn;
2. giải mã lại CAVLC, thu ứng viên (giống phía gửi vì lật dấu không đổi cấu trúc);
3. dẫn xuất khóa con, xếp lịch HMAC của đoạn, đọc bit dấu tại vị trí đã chọn;
4. bỏ làm trắng 24 bit đầu, kiểm version `0x03` và độ dài, biết tổng số bit cần đọc;
5. đọc đủ khung qua các đoạn, kiểm định dạng; in payload hex hoặc thoát mã 2.

Payload in ra lúc này **chưa được xác thực**; chỉ bước Groth16 dưới đây mới quyết định.

9.2 Unpack, hash video và verify Groth16

`src/camera_proof.py::verify_payload()` nhận `bridge`, `root`, `payload`, CLI native, file stego, khóa `K` và `max_bits_per_idr`:

1. `parse_payload()`: `unpack_payload()` (đúng format, mode, độ dài chính xác) rồi `bytes_to_proof()`; lỗi → `malformed_proof_payload`.
2. `check_proof()`: mode 1 bị từ chối với `video_binding_required` khi `require_video_binding` (mặc định); mode 0 thì `native_video_digest()` tính lại digest từ *chính file nhận được* với `frame_bits_for_message(len(m))`; lệnh lỗi → `video_digest_unavailable`.
3. `binding_digest(mode, digest, m)` rồi `CameraProofBridge.verify()` với (`proof`, `root`, `binding`); sai → `proof_invalid`.
4. Hợp lệ → `Verdict` có `valid=True`, mode và message; thuộc tính `video_bound` đúng khi mode = 0. Message chỉ được dùng sau bước này.

Demo (`verify_message()`) còn kiểm thêm digest của stego bằng digest của cover và dựng lại lịch từ stego bằng `segment_schedule()`; đó là kiểm toán giáo dục, không phải input của quyết định.

9.3 Verifier phía API

`native_handlers.make_proof_verify_handler(extract_payload, check)` trả về:

Kết quả	Điều kiện
<code>valid: true, message_b64, payload_bytes, video_bound</code>	Trích được payload, unpack đúng, digest tính được, proof verify với root. Job → <code>succeeded</code> .
<code>reason: payload_not_found</code>	CLI thoát khác 0 (<code>NativePayloadNotFound</code>): không có khung v3 dưới khóa này (sai khóa, video đã mã hóa lại, mất IDR mang bit). Job → <code>rejected</code> .
<code>reason: malformed_proof_payload</code>	<code>parse_payload()</code> trả None.
<code>reason: video_binding_required</code>	Payload mode 1 khi chưa bật <code>ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1</code> .
<code>reason: video_digest_unavailable</code>	Lệnh <code>video-digest</code> thất bại trên file tải lên.
<code>reason: proof_invalid</code>	Groth16 trả False (message bị đổi, video bị sửa/cắt, payload chép sang video khác, camera ngoài sổ đăng ký...).
Ngoại lệ hạ tầng	Timeout, thiếu Node.js, verification key hoặc sổ đăng ký (<code>ZK_STEGO_CAMERA_REGISTRY</code>) → job failed, không phải rejected.

10 API: hàm nào nhận request, hàm nào chạy job

10.1 Lỗi job chung `src/api/app.py`

`create_app(settings, embed_handler=, verify_handler=, extract_handler=)` đăng ký endpoint; endpoint có handler None trả 501. Ứng dụng native inject đủ cả ba.

Hàm/lớp	Input → output	Ảnh hưởng
ApiSettings.from_environment()	env → settings	ZK_STEGO_API_TOKEN bắt buộc, tối thiểu 32 ký tự (MIN_API_TOKEN_LENGTH); work dir, circuits dir (ZK_STEGO_CIRCUITS_DIR), số đăng ký camera (ZK_STEGO_CAMERA_REGISTRY), require_video_binding (tắt chỉ khi ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1), timeout upload, thời gian giữ job, ZK_STEGO_API_DOCS.
RequestBodyGuard.__call__()	ASGI scope → request được bảo vệ	Kiểm Bearer token trước khi đọc body , giới hạn kích thước/thời gian upload, tối đa 4 upload đồng thời.
AuthFailureLimiter	IP → retry_after()	10 lần sai trong 60 s → 429 kèm Retry-After; dùng chung cho HTTP và WebSocket.
bearer_token_matches(), require_token()	header → pass/401	So token; require_token() là lớp phòng thủ thứ hai.
WorkDirLease.acquire()/release()	work dir → khóa	Ngăn hai instance dùng chung thư mục.
JobStore.create/get/update/discard()	record ↔ JSON	Không lưu secret hay nội dung message.
_decode_key(), _decode_message()	base64 → bytes	Kiểm định dạng trước khi xếp job.
_save_upload()	UploadFile → file	Ghi có giới hạn; lỗi thì dọn.
reserve_capacity()	semaphore → slot/503	Mặc định 1 worker + 2 job chờ.
submit(), outcome()	job id, operation, callable	Chạy nền; job verify có valid khác True → rejected; ngoại lệ → failed. Trạng thái: queued, running, succeeded, rejected, failed.
start_embed()	video, message_b64, secret_key_b64	Gọi embed_handler; message_b64 là blob opaque (demo gửi payload mang proof đã đóng gói), server không tạo proof.
start_extract()	stego_video, khóa, maximum_payload_bytes (1..4096)	Gọi extract_handler; artifact payload.bin; kết quả mang "verified": false (_safe_extract_result()).

Hàm/lớp	Input → output	Ảnh hưởng
start_verify()	stego_video, khóa, maximum_payload_bytes	Gọi verify_handler; không cần video gốc; _safe_verify_result() chỉ trả message khi proof đúng.
get_job(), download_artifact()	job id → trạng thái/file	Artifact tải một lần , hết hạn sau 10 phút.
startup(), shutdown(), recover_interrupted_jobs(), purge_expired_jobs()	vòng đời	Khôi phục job đang dở, dọn job quá hạn (mặc định 24 h).

10.2 Ứng dụng native src/api/native_handlers.py

create_native_app() đọc ZK_STEGO_NATIVE_CLI (đường dẫn zkstego_blind_bits) và ZK_STEGO_MAX_BITS_PER_IDR (mặc định 64), rồi định nghĩa:

- invoke(): chạy CLI không qua shell, có timeout; stdin = khóa hex + phần đuôi; mã thoát khác 0 → NativePayloadNotFound.
- embed_handler(): embed-stream-auth-stdin; bits_embedded được *tính* là $(|m| + 3) \times 8$ (FRAME_HEADER_BYTES=3, bits_embedded_source=computed_frame_size), không do CLI báo.
- native_extract_payload() và extract_handler(): extract-stream-auth ... -, kiểm tra hex và giới hạn payload, ghi file mới.
- camera_check(): nạp CameraRegistry.load() từ settings.camera_registry_path và CameraProofBridge(settings.circuits_dir), rồi gọi camera_proof.check_proof() với CLI native, file tải lên và khóa. Thiếu số đăng ký hoặc verification key là lỗi triển khai (failed).
- make_proof_verify_handler(native_extract_payload, camera_check): handler của job verify; test có thể inject payload_extractor, proof_checker.
- native_stream(): WebSocket /api/v1/stream.

Giao thức WebSocket: xác thực *trước* accept() (thiếu token thì handshake bị từ chối). Trình tự thông điệp:

```
client -> {"operation":"embed","secret_key_b64":...,"message_b64":...}
        or {"operation":"extract","secret_key_b64":...,"maximum_payload_bytes":N}
server -> {"type":"ready","operation":...}
client -> binary Annex-B chunks ..., then {"type":"end"}
server -> embed: binary H.264 chunks, then {"type":"complete",...}
        extract: {"type":"payload","payload_b64":...,"verified":false}
        error:   {"type":"error","code":"payload_not_found" | "stream_idle_timeout"
" | ...}
```

Payload của luồng live chưa được xác thực; luồng live chỉ có thể mang proof mode 1 (chỉ ràng buộc message) vì phần sau của video chưa tồn tại lúc nhúng. Server giới hạn byte/chunk, đặt high/low water mark stdin của tiến trình con (64/16 KiB), đọc stderr liên tục và đóng luồng sau ZK_STEGO_STREAM_IDLE_TIMEOUT_SECONDS (15 s) không có dữ liệu. Đây là dịch vụ trên elementary stream H.264; module này không tự mở thiết bị camera.

11 Các module hỗ trợ ngoài đường embed/extract

11.1 Bộ lập lịch realtime

src/realtime_cavlc.py: CAVLCRealtimeBudget.__post_init__() kiểm FPS, giới hạn hàng đợi và bộ nhớ; RealtimeCAVLCScheduler.submit() nhận một đoạn với epoch tăng đơn điệu, bỏ đoạn cũ nhất khi đầy; process_next() lấy proof qua _proof_for() (cache tối đa 4 epoch), gọi callback patch, đo độ trễ; report() tính số đoạn bị bỏ, failed_segments, p95 và accepted. Đây là abstraction được test riêng; WebSocket native tự quản lý tiến trình con và backpressure, không gọi scheduler này. Đo trên camera thật **chưa** đạt công 30 FPS: lần chạy hiệu chỉnh ngày 24/09/2026 (giao thức v1, trước khi tăng tốc native ngày 02/10/2026) đạt 7,608 FPS và chưa được đo lại.

11.2 Chứng chỉ khóa và manifest

src/key_policy.py::issue_key_certificate() ký chứng chỉ Ed25519 (khóa công khai chủ thể, key id, issuer, thời điểm bắt đầu/hết hạn); KeyCertificate.verify() kiểm chữ ký, định dạng và cửa sổ thời gian; verify_manifest_with_certificate() kiểm chứng chỉ rồi verify manifest. src/manifest.py::StegoManifest có sign(), verify_signature(), compute_content_hash(), to_json()/from_json(). Các helper này chỉ có hiệu lực nếu được gọi tại điểm chấp nhận khóa; đường native hiện nhận khóa thô 32 byte và không gọi KeyCertificate.verify().

11.3 Giao diện trust thử nghiệm

src/trust/ gồm provenance.py, fingerprint.py, attestation.py, watermark_receipt.py, canonical.py và c2pa_bridge.py (build_c2pa_anchor(), verify_c2pa_anchor()). Các mạch circuits/fingerprint_verify.circom và circuits/detector_receipt.circom thuộc cùng hướng thử nghiệm. Chúng được kiểm bởi test_trust_interfaces.py nhưng không nằm trên đường runtime của demo hay API.

12 Kiểm thử và tái lập

```
py -3.12 -m pip install -r requirements.txt
cd circuits; npm ci; cd ..
cmake --build native/build --config Release
py -3.12 src/runtest/prepare_fixtures.py      # regenerate data/encoded/*.h264 from
data/raw
py -3.12 src/runtest/run_all.py              # all 12 phases
py -3.12 src/runtest/run_all.py --quick      # fast phases only
ctest --test-dir native/build -C Release --output-on-failure
```

Phase	File trong src/runtest/	Nội dung
1	test_zk_proof.py	Định dạng proof/payload, vector Poseidon, số đăng ký, binding, đối chiếu witness của mạch, Groth16 thật
2	test_native_blind_contract.py	C++ và Python khớp HKDF, lịch, khung v3, làm trắng, lịch theo đoạn
3	test_native_cli_fixture.py	CLI nhúng/trích trên fixture, decode nghiêm ngặt, sai khóa; video-digest native = Python, cover = stego

Phase	File	Nội dung
4	test_h264_tables.py	Bảng VLC (prefix-free, Kraft, giá trị chuẩn)
5	test_service_api.py	Lỗi job, xác thực trước body, giới hạn thử sai, job verify
6	test_native_http_channel.py	Job HTTP và WebSocket qua Uvicorn thật; proof ràng buộc video được chấp nhận, chép sang video khác và cắt ngắn bị từ chối
7	test_manifest_security.py	Ký manifest và ràng buộc liên quan
8	test_realtime_scheduler.py	Bộ lập lịch realtime
9	test_benchmark_recorder.py	Fail-closed của bộ ghi benchmark camera
10	test_trust_interfaces.py	Provenance, C2PA, attestation thử nghiệm
11	test_demo_h264_explain.py	Toán giải thích của demo khớp native và pixel FFmpeg
12	test_demo_smoke.py	Toàn bộ demo chạy không có [FAIL]
H1	test_native_camera_http.py	Camera thật, chỉ khi --hardware

Mã thoát của run_all.py: 0 = tất cả qua, 1 = có lỗi, 2 = có phase thiếu bằng chứng (bị bỏ qua). Lần chạy ngày 04/10/2026 đạt **142/142** qua 12 phase và CTest native 1/1; phase camera H1 không nằm trong con số này.

12.1 Số liệu hiện tại để đối chiếu

Lần đo ghi nhận ngày 04/10/2026 (i7-12700H, Windows 11, giao thức v3 + proof camera; chi tiết trong benchmark/NEW_BENCHMARKS.md và benchmark/results/e2e_new.json):

Hạng mục	Kết quả
Media	Run 20261004T142448Z_1db00a: chuyển đổi 9/9, media 27/27 (9 clip CIF $\times$ 352 $\times$ 288, 640 $\times$ 480, 1280 $\times$ 960; QP 22, all-intra, 30 frame đầu; mọi proof ràng buộc video; sổ đăng ký 8 camera); không dấu nào ngoài lịch bị đổi.
E2E (clip CIF 300 frame, trung vị, 5/5)	Digest cover 0,51 s; Groth16 prove 1,77 s; nhúng 0,50 s; decode nghiêm ngặt 0,15 s; trích 0,53 s; verify (digest stego + Groth16) 1,45 s; tổng phía gửi 2,78 s, phía nhận 1,99 s.
Tấn công	11/11 đúng kỳ vọng: sai khóa và mã hóa lại $\rightarrow$ payload_not_found; lật dấu carrier $\rightarrow$ malformed_proof_payload; lật dấu không mang khung, sửa frame sau, cắt nửa video, đổi message, camera ngoài sổ đăng ký, chép payload sang video khác $\rightarrow$ proof_invalid.
ZKP	Groth16 prove 1,5–1,8 s, verify 0,9–1,0 s, proof 129 byte; PLONK prove khoảng 22 s, proof khoảng 2,25 KB JSON.
Chi phí native	Trung vị quét dung lượng mỗi IDR 52 / 220 / 770 ms, nhúng mỗi IDR mang bit 61 / 245 / 891 ms, hash video (26 IDR carrier) 1,6 / 6,4 / 22,6 s ở ba độ phân giải; parse, không phải Groth16, giới hạn thông lượng.
Chất lượng	PSNR-Y cover $\rightarrow$ stego thấp nhất mỗi ca 44,03–58,68 dB, 0/27 dưới 40 dB; kích thước stego bằng cover.

13 Demo đã chạy: artifact và cách suy luận

Artifact	Nội dung	Dùng để kết luận gì
transcript.txt, report.json	Thuyết minh (secret bị che), kết quả kiểm tra, thời gian	Trạng thái tổng; luôn mở log tiến trình gốc khi có lỗi.
encode.stderr, encoder_input.yuv, source_decoded.yuv	Log FFmpeg/libx264, frame trước/sau nén	Mất mát do nén, tách biệt với nhúng.
trace_before.json, trace_after.json	NAL, slice, ứng viên, bit dấu, hệ số	Chỉ đúng các bit đã lập lịch đổi; cấu trúc giữ nguyên.
segments_before.json, segments_after.json	Ứng viên từng đoạn (segments-1)	Bên nhận dựng lại cùng lịch từ stego.
schedule.json	Bit khung, keystream, bit nhúng, cũ/mới từng vị trí	Kiểm lịch HMAC và làm trắng.
proof.json, proof.bin, public_signals.json	Proof và 3 public signal [root, bindingHi, bindingLo]	Kiểm nén/giải nén proof.
camera_registry.json	Sổ đăng ký 4 camera demo (khóa công khai, root)	Root mà verifier và job verify của demo tin.
mb_detail_before.json, mb_detail_after.json	Output --macroblock	Giải phẫu khối mẫu.
invalid_proof.h264, missing_proof.h264, tampered_carrier.h264, cut_last_frame.h264	Video của các ca từ chối	Mỗi ca bị từ chối đúng lý do (proof_invalid, malformed_proof_payload, khung không tìm thấy).
native_embed.*, native_extract.*, reject_wrong_key.*	stdout/stderr từng tiến trình	Mã thoát 2 và thông báo lỗi đúng lý do.

PASS của demo là bằng chứng chức năng trên một clip/một lần chạy, không phải benchmark: nó không đo chất lượng cảm nhận trên tập lớn và không chứng minh deadline realtime.

14 Các giới hạn cần nhớ khi đọc code

- Không phải mọi H.264 đều được hỗ trợ.** Parser native chỉ nhận Baseline, CAVLC, progressive, 4:2:0, một slice mỗi frame bắt đầu ở MB 0, không I_PCM, không nhiều SPS/PPS khác id.
- Chỉ IDR mang bit.** Với GOP > 1, P-frame được sao chép nguyên.
- Mù không có nghĩa không cần khóa.** Extractor không cần video gốc nhưng cần khóa giấu tin và cùng max_bits_per_IDR. Hash video cũng phụ thuộc khóa này (vị trí carrier), nên chỉ người có khóa mới kiểm chứng được.
- Groth16, HMAC và bearer token khác nhiệm vụ.** Proof là thứ duy nhất xác thực payload; HMAC chỉ là hàm giả ngẫu nhiên có khóa để chọn vị trí và làm trắng; token xác thực caller của API. Payload từ lệnh/job extract và từ luồng live mang "verified": false.
- Luồng live chỉ ràng buộc message (mode 1),** vì phần sau của video chưa tồn tại khi nhúng proof; job verify từ chối mode 1 trừ khi bật cấu hình.
- Setup Groth16 Phase 2 là nghi thức demo** trên một máy (beacon sinh cục bộ); triển khai thật cần nhiều bên độc lập.
- Proof không chứng minh camera thật sự quay cảnh đó** (ví dụ quay lại màn hình); nó chỉ chứng minh một camera trong sổ đăng ký đã xác nhận đúng video và message.

8. **Keystream làm trắng không có nonce:** cùng khóa thì dùng lại cùng dòng keystream; chưa có chứng minh hay đo đặc khả năng chống steganalysis.
9. **Mong manh theo thiết kế.** Mọi lần mã hóa lại đều phá kênh; realtime trên camera thật chưa đạt công 30 FPS (xem `future_plan.md`).

Các tấn công đã được kiểm và bị từ chối: chép payload sang video khác, cắt video, sửa frame sau, lật dấu không mang khung, mã hóa lại, đổi message, camera ngoài sổ đăng ký, sai khóa.

15 Lộ trình đọc source cho người mới tiếp nhận

1. Đọc `doc/he_thong_hoat_dong.md` và `demo/README.md`, chạy `py -3.12 demo/terminal_demo.py --auto`, mở thư mục `demo/runs/terminal_*`.
2. Đọc `terminal_demo.py::walkthrough()` theo bảng ở Mục 2.2, rồi `demo/native_io.py` để thấy Python chỉ gọi tiến trình native.
3. Đọc `native/tools/blind_bits.cpp::run()`, sau đó `embed_cavlc_stream_file()`, `extract_cavlc_stream_file()` và `cavlc_video_binding_digest()`.
4. Đọc thư viện theo chuỗi `split_annex_b() → decode_baseline_i_idr_slices() → collect_cavlc_trailing_one_sign_candidates() → prepare_cavlc_stream_segment() → select_with_schedule_key() → CavlcStreamEncoder / CavlcStreamDecoder`.
5. Đối chiếu với `src/native_blind_contract.py`, `src/video_binding.py` và `test_native_blind_contract.py`.
6. Đọc proof path: `src/camera_registry.py`, `circuits/camera_video.circom`, `src/zk_proof.py`, `src/camera_proof.py` và `src/zk_setup.py`.
7. Đọc boundary dịch vụ: `native_handlers.create_native_app()`, `make_proof_verify_handler()`, `native_stream()` và `app.create_app()`.

16 Tra cứu nhanh theo câu hỏi debug

Triệu chứng	Nơi bắt đầu lần theo
Không thấy IDR / parser từ chối	<code>zkstego_inspect --summary</code> ; <code>split_annex_b()</code> , <code>parse_baseline_sps()/pps()</code> , <code>decode_baseline_i_idr_slices()</code> ; kiểm profile và <code>slices=1</code> .
Dung lượng thấp, payload không vừa	<code>zkstego_inspect --segments N</code> ; <code>segment_plan.resolve_max_bits()</code> ; tăng <code>max_bits_per_IDR</code> , số frame hoặc giảm message.
Embed thoát mã 2	<code>stderr</code> của <code>native_embed</code> ; <code>embed_cavlc_stream_file()</code> , <code>process_segment()</code> .
FFmpeg từ chối video	Kiểm ranh giới patch/EBSP: <code>patch_annex_b_rbsp_plan()</code> , <code>rbasp_to_ebsp()</code> .
Extract báo sai version/độ dài	Cùng khóa hai phía? Cùng <code>max_bits_per_IDR</code> ? Stego v2? Rồi <code>consume_segment()</code> , <code>unpack_frame()</code> .
Lịch Python khác native	<code>segment_schedule()</code> so với <code>analyze_cavlc_stream_file()</code> ; nhớ <code>nal_index</code> tính trong input phân tích của đoạn.

Triệu chứng	Nơi bắt đầu lần theo
Digest cover khác digest stego	Cùng khóa, cùng frame_bits (frame_bits_for_message()) và cùng cap? So video_binding.video_digest() với lệnh video-digest.
Payload trích được nhưng ZK false	Kiểm header payload 4 byte, proof 129 byte, digest của file nhận, root của sổ đăng ký và camera_video_vkey.json; xem check_proof(), _snarkjs_verify().
Job verify rejected	Đọc reason: payload_not_found, malformed_proof_payload, video_binding_required, video_digest_unavailable, proof_invalid.
Job verify failed	Lỗi hạ tầng: Node.js, ZK_STEGO_CIRCUITS_DIR, ZK_STEGO_CAMERA_REGISTRY, verification key, timeout.
HTTP 401/429	RequestBodyGuard, AuthFailureLimiter; token phải ≥ 32 ký tự.
Khóa hết hạn vẫn dùng được	Đường native nhận khóa thô; KeyCertificate.verify() chưa được gọi ở đó.

17 Kết luận vận hành

Khi đọc một đường lỗi, lần theo artifact tuần tự: message, digest của cover, binding, proof 129 byte, payload, khung v3 (đã làm trắng), lịch theo đoạn, video stego, payload trích được, message và proof sau unpack, digest của stego, rồi kết quả Groth16 với root. Điểm nổi quan trọng nhất là CLI native nhận payload là byte opaque: Python dựng sổ đăng ký, binding, proof và payload; phân tích cú pháp H.264, khóa con, lịch, khung, làm trắng, patch, hash video và trích mù đều nằm trong C++. Khi sửa một hàm, kiểm đúng consumer kế tiếp của nó và chạy test ở boundary tương ứng (CTest cho lỗi, phase 1 cho proof, sổ đăng ký và binding, phase 2 cho hợp đồng chéo, phase 3/5/6 cho CLI và dịch vụ).

Ghi chú lịch sử. Trước ngày 03/10/2026, repository còn một nhánh Python full-video (src/embedder.py, src/verifier*.py, src/core/, src/bitstream/, chaos map, sidecar positions.json, cache pickle) và bộ demo nhiều script đánh số; nhánh này và các benchmark SEC1–SEC10 đã bị gỡ (vẫn xem được trong lịch sử git), còn c2pa_bridge.py được chuyển từ src/provenance/ sang src/trust/.